



AUTOMATED MACHINE LEARNING (AUTOML) PLATFORM WITH DYNAMIC PIPELINE ROUTING

A Project Report

Submitted in partial fulfilment of the requirements for the degree of

**Bachelor of Technology
in
Computer Science and Engineering**



Submitted by

Jamil Aktar	13100123029
Jayed Islam	13100123030
Dilruba Khatun	13100123028

Under the Guidance of

Kapil Dev Das

Department of Computer Science and Engineering

Department of Computer Science and Engineering



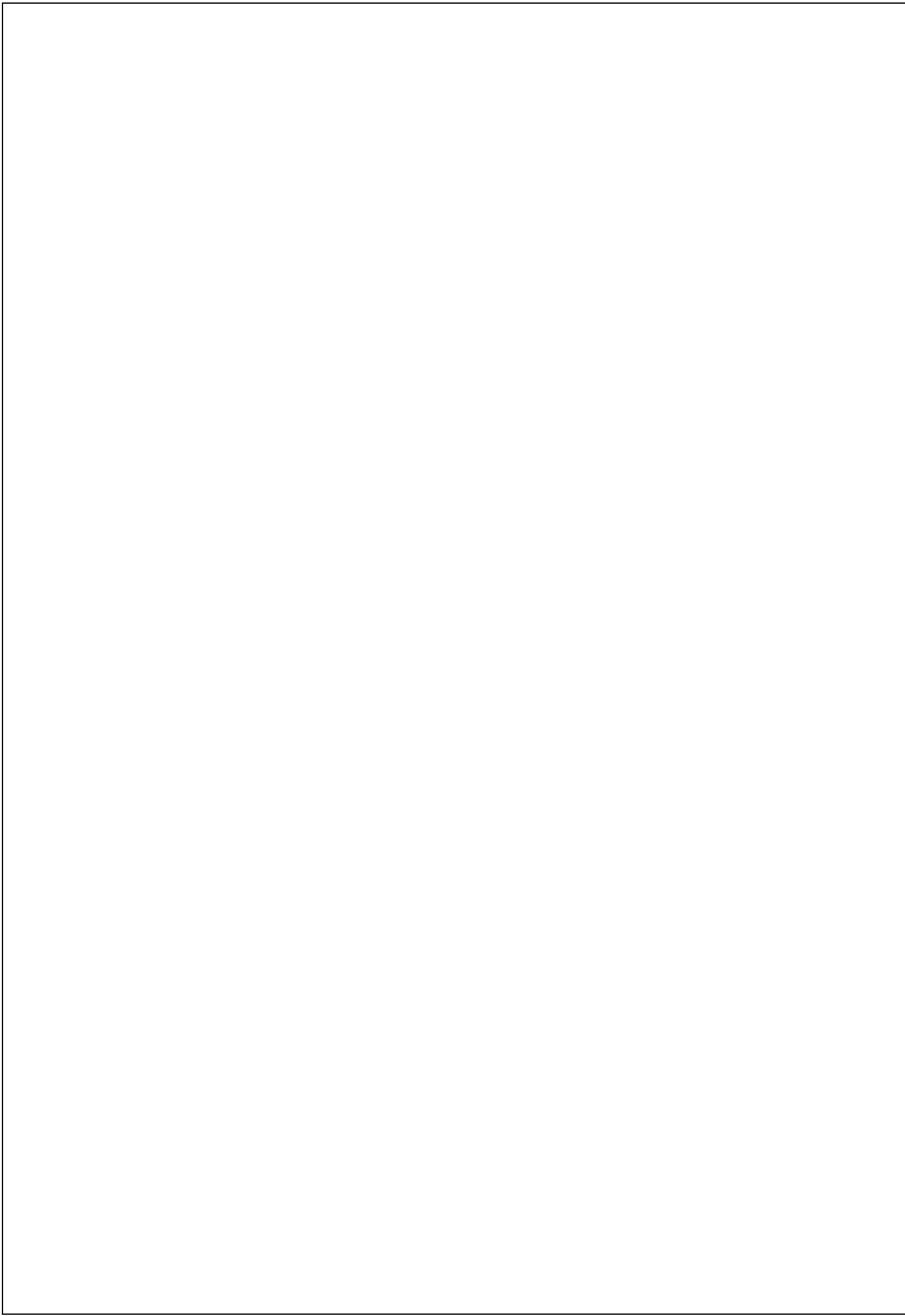
DOMKAL INSTITUTE OF ENGINEERING AND TECHNOLOGY

Domkal, Murshidabad, West Bengal - 742406

Affiliated to

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY (MAKAUT)

2026



Certificate of Approval

This is to certify that the project report entitled "**Automated Machine Learning (AutoML) Platform with Dynamic Pipeline Routing**" submitted by **Jamil Aktar, Jayed Islam, Dilruba khatun** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from Maulana Abul Kalam Azad University of Technology (MAKAUT), is a bona fide record of the project work carried out by him under my supervision and guidance.

To the best of my knowledge, the matter embodied in this project report has not been submitted to any other University or Institute for the award of any other degree or diploma.

Signature of the Guide

Name: Kapil Dev Das

Designation: Assistant Professor

Department of Computer Science and Engineering

Domkal Institute of Engineering and Technology

Declaration

I, **Jamil Aktar, Jayed Islam, Dilruba Khatun**, hereby declare that the project report entitled "**Automated Machine Learning (AutoML) Platform with Dynamic Pipeline Routing**" submitted in partial fulfilments of the requirements for the degree of **Bachelor of Technology in Computer Science and Engineering** at Maulana Abul Kalam Azad University of Technology (MAKAUT), is an authentic record of my own original work carried out at Domkal Institute of Engineering and Technology under the supervision and guidance of **Kapil Dev Das**

I further declare that the work reported in this project has not been submitted, either in part or in full, to this or any other University or Institution for the award of any other degree, diploma, or fellowship.

Name	Roll No	Signature
Jamil Aktar	13100123029	
Jayed Islam	13100123030	
Dilruba Khatun	13100123028	

Place: Domkal, Murshidabad

Date: 16.07.2026

4. Acknowledgements Page

The successful completion of this final year project would not have been possible without the support, guidance, and encouragement of several individuals. I would like to take this opportunity to express my deepest gratitude to all of them.

First and foremost, I express my sincere gratitude to my project guide, **Kapil Dev Das**, for their continuous support, invaluable technical insights, and patience throughout the development of this AutoML platform. Their mentorship was instrumental in shaping this project into its final form.

I would also like to extend my profound thanks to **Suman Pal**, Head of the Department of Computer Science and Engineering, for providing the necessary resources, laboratory facilities, and an encouraging environment to carry out this research.

My sincere thanks go to our Honourable Principal, **Prof (Dr) Md Headayayetullah**, for the continuous motivation and administrative support provided by the institution.

I also wish to thank all the faculty members and technical staff of the Computer Science and Engineering department at Domkal Institute of Engineering and Technology for their direct and indirect assistance during my B.Tech journey.

Finally, I am deeply indebted to my parents, family members, and friends for their unwavering moral support, sacrifices, and encouragement throughout my academic career.

SYNOPSIS

1. Introduction and Objective

The rapid proliferation of data-driven decision-making has catalysed the demand for Machine Learning (ML) solutions across various domains. However, developing robust predictive models requires specialized expertise in statistical modelling, feature engineering, and algorithmic optimization. The primary objective of this project is to democratize machine learning by developing an Automated Machine Learning (AutoML) platform. This system abstracts the underlying computational complexities, allowing users to upload raw, heterogeneous tabular datasets and automatically obtain optimized, production-ready machine learning models alongside interactive analytical reports.

2. Problem Formulation

Traditional predictive modelling is characterized by highly iterative, manual workflows encompassing data sanitization, categorical encoding, and hyperparameter tuning. Existing commercial AutoML platforms often introduce vendor lock-in and prohibitive costs, while open-source alternatives primarily exist as programmatic libraries lacking accessible graphical interfaces. This project addresses the critical need for a scalable, open-architecture web system capable of autonomously navigating the end-to-end ML lifecycle—handling dirty data, selecting appropriate algorithms, and preventing UI-blocking computational bottlenecks.

3. Proposed Methodology and Architecture

The project utilizes a distributed, microservice-based architecture to ensure high availability and responsiveness during computationally intensive ML tasks. The core methodologies include:

- **Algorithmic Routing Engine:** The system dynamically evaluates the schema of the uploaded dataset. It utilizes target-variable inspection to autonomously route tasks to Supervised Learning pipelines (Regression for continuous variables, Classification for discrete variables) or Unsupervised Learning pipelines (K-Means Clustering) when no target is provided.
- **Dynamic Preprocessing & Feature Engineering:** The backend constructs automated Scikit-Learn pipelines that handle data imperfections programmatically. This includes median imputation for missing numerical matrices and Most-Frequent imputation followed by One-Hot Encoding for categorical string arrays.
- **Asynchronous Task Orchestration:** To prevent HTTP request timeouts and server thread blocking, ML workloads are decoupled from the main web server. The Flask API delegates training tasks to a Celery distributed worker using Redis as an in-memory message broker, allowing the frontend to poll for asynchronous progress updates.
- **Automated Exploratory Data Analysis (EDA):** Concurrent with model training, the system executes statistical data profiling (via ydata-profiling) to generate interactive, browser-native HTML reports and PowerBI-compatible JSON payloads containing correlation matrices and distribution analyses.

4. Technological Ecosystem

- **Client-Side Presentation Layer:** Next.js (React), Tailwind CSS
- **API & Orchestration Layer:** Python, Flask, Flask-CORS
- **Asynchronous Message Queue:** Redis, Celery
- **Machine Learning & Analytical Engine:** Scikit-Learn, Pandas, NumPy

- Infrastructure & Containerization: Docker, Docker Compose

5. Expected Outcomes and Deliverables

1. Distributed Web Application: A fully functional, responsive client interface with real-time polling mechanisms and state management.
2. Production-Ready Artifacts: Automated generation and serialization of .joblib pipelines, encapsulating both the optimal ML estimator and the required data preprocessing steps for seamless deployment.
3. Automated Analytical Reporting: Dynamic generation of interactive statistical reports for immediate exploratory analysis.
4. Containerized Environment: A modular, Dockerized architecture demonstrating modern DevOps practices and ensuring cross-platform reproducibility.

6. Future Scope

The foundational architecture developed in this project can be expanded to support out-of-core distributed computing frameworks (e.g., Apache Spark or Dask) to process datasets exceeding standard memory limits. Furthermore, the routing engine can be extended to support Natural Language Processing (NLP) for unstructured text data and Deep Neural Networks (DNNs) for complex sequential analysis.

Abstract

The rapid integration of predictive analytics into modern business applications has driven an unprecedented demand for Machine Learning (ML) solutions. However, the traditional ML lifecycle remains a highly manual, iterative process that requires deep technical expertise in data preprocessing, feature engineering, and algorithm optimization, thereby creating a significant barrier to entry for non-technical domain experts. This project presents a scalable, full-stack Automated Machine Learning (AutoML) platform designed to democratize access to predictive modelling by completely abstracting the computational complexities of data science.

The developed system employs a distributed microservice architecture utilizing Next.js for a responsive client interface, alongside Flask, Celery, and Redis for asynchronous task orchestration. This decoupling ensures that intensive ML workloads do not block the web server, allowing for real-time UI updates. At its core, the platform features an intelligent routing engine that autonomously inspects raw tabular data to handle missing values and categorical encodings programmatically. It dynamically selects and evaluates Supervised Learning (Classification and Regression) or Unsupervised Learning (K-Means Clustering) algorithms based on the user-defined target variable's data type. Concurrently, the platform integrates automated Exploratory Data Analysis (EDA) to generate interactive HTML profiling reports and PowerBI-compatible JSON payloads. The final output is a fully serialized, production-ready Scikit-Learn .joblib pipeline. Ultimately, this project demonstrates a modern, Docker-containerized solution that significantly reduces the time-to-deployment for machine learning models while providing deep, accessible analytical insights.

Table of Contents

Title	Page No.
Certificate of Approval	i
Student Declaration	ii
Acknowledgements	iii
Synopsis	iv-v
Abstract	vi
Table of Content	vii - viii
List of Abbreviations	ix
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Proposed Solution	1
1.4 Scope of the Project	2
CHAPTER 2: LITERATURE REVIEW	2
2.1 Existing Systems (Google AutoML, H2O.ai, Auto-Sklearn)	2
2.2 Limitations of Existing Commercial & Open-Source Systems	2
2.3 Motivation for the Proposed Architecture	3
CHAPTER 3: SYSTEM ARCHITECTURE & TECHNOLOGY STACK	4
3.1 High-Level Microservice Architecture	4
3.2 Client-Side Presentation Layer (Next.js & Tailwind CSS)	5
3.3 API & Orchestration Layer (Python & Flask)	6
3.4 Asynchronous Task Processing (Celery & Redis)	6
3.5 Infrastructure & Containerization (Docker)	7
CHAPTER 4: IMPLEMENTATION & METHODOLOGY	7
4.1 Automated Exploratory Data Analysis (EDA) & Profiling	7
4.2 Dynamic Preprocessing & Feature Engineering	8
4.2.1 Missing Value Imputation	9
4.2.2 Categorical One-Hot Encoding	9
4.3 Intelligent Algorithmic Routing Engine	9
4.3.1 Supervised Learning (Classification & Regression)	10

4.3.2 Unsupervised Learning (K-Means Clustering)	11
4.4 Model Serialization and Deployment Preparation (.joblib)	11
CHAPTER 5: RESULTS & VISUALIZATIONS	11
5.1 User Interface & State Management Dashboard	11
5.2 Testing Scenarios & Execution Outputs	13
5.2.1 Case 1: Continuous Target (Regression Route)	13
5.2.2 Case 2: Discrete Target (Classification Route)	13
5.2.3 Case 3: No Target (Clustering Route)	14
5.3 HTML EDA Reports & Power BI JSON Integration	14
CHAPTER 6: CONCLUSION & FUTURE SCOPE	15
6.1 Conclusion	16
6.2 Future Enhancements	15 - 16
REFERENCES / BIBLIOGRAPHY	18
APPENDIX (Source Code Snippets & Setup Guide)	19-25

List of Abbreviations

API	Application Programming Interface
AutoML	Automated Machine Learning
AWS	Amazon Web Services
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
GCP	Google Cloud Platform
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
NLP	Natural Language Processing
OOM	Out of Memory
RAM	Random Access Memory
REST	Representational State Transfer
SaaS	Software as a Service
UI	User Interface
UUID	Universally Unique Identifier
XAI	Explainable Artificial Intelligence

CHAPTER 1: INTRODUCTION

1.1 Background

In the contemporary digital era, data is generated at an unprecedented scale across almost all sectors, including finance, healthcare, e-commerce, and manufacturing. The extraction of actionable insights and predictive patterns from this data is fundamentally driven by Machine Learning (ML). Consequently, organizations are increasingly reliant on predictive analytics to forecast trends, classify behaviors, and optimize decision-making processes.

However, the traditional lifecycle of developing a machine learning model is highly complex and multidisciplinary. A standard data science workflow requires extensive programmatic and statistical knowledge to navigate several iterative phases: data ingestion, exploratory data analysis (EDA), data cleaning (handling missing values and outliers), feature engineering (categorical encoding and scaling), algorithm selection, hyperparameter tuning, and deployment. Historically, these steps required manual execution using languages like Python or R, restricting the power of machine learning to a small subset of highly trained software engineers and data scientists.

1.2 Problem Statement

Despite the proliferation of powerful open-source ML libraries like Scikit-Learn, the practical application of machine learning remains hindered by a significant barrier to entry. Domain experts—such as business analysts, medical researchers, and financial managers—often possess valuable datasets but lack the coding proficiency required to build predictive models.

Furthermore, even for experienced developers, the manual preparation of data is a major bottleneck. Real-world tabular data is inherently "dirty." Datasets frequently contain missing numerical cells, varying scales, and unencoded categorical strings. Standard ML algorithms immediately fail when fed this raw data. An engineer must manually write boilerplate code to impute missing values, apply One-Hot Encoding, and test multiple algorithms (e.g., deciding between a Random Forest or Logistic Regression) to find the optimal fit. This repetitive, manual workflow is time-consuming, prone to human error, and lacks scalability.

1.3 Proposed Solution

To address these challenges, this project proposes the development of a full-stack, web-based Automated Machine Learning (AutoML) platform. The primary objective is to democratize machine learning by completely abstracting the complex data science pipeline behind an intuitive, "no-code" graphical user interface.

The developed platform enables users to simply drag and drop a raw CSV dataset into a Next.js web dashboard and specify a target variable they wish to predict. Once uploaded, the system utilizes a distributed microservice architecture (powered by Flask, Celery, and Redis) to securely process the data in the background without freezing the user interface.

The core intelligence of the platform lies in its dynamic routing and automated preprocessing engine. The system autonomously inspects the dataset to handle missing values and encode text into machine-readable formats. It then intelligently determines the nature of the predictive task—routing to Regression algorithms for continuous data, Classification algorithms for discrete data, or K-Means Clustering if the user provides no target at all. Concurrently, the platform automates Exploratory Data Analysis (EDA), generating interactive HTML and PowerBI-ready JSON reports to provide immediate statistical insights to the user.

1.4 Scope of the Project

The scope of this project is strictly bounded to ensure high performance, reliability, and architectural stability. The system focuses on traditional machine learning applied to structured, tabular datasets.

In-Scope Functionalities:

- **Supported File Types:** Comma-Separated Values (CSV).
- **Supervised Learning:** Automated pipeline generation for Classification (Logistic Regression, Random Forest Classifier) and Regression (Linear Regression, Random Forest Regressor).
- **Unsupervised Learning:** Automated K-Means Clustering when no target variable is designated.
- **Automated Preprocessing:** Median imputation for numerical gaps, most-frequent imputation, and One-Hot Encoding for categorical text.
- **Exportable Artifacts:** Downloading the final, production-ready pipeline as a serialized Scikit-Learn .joblib file, alongside interactive HTML profiling reports.

Out-of-Scope Limitations:

- The current architecture does not support unstructured data such as raw images, audio, or video files.
- Deep Learning frameworks (such as TensorFlow or PyTorch) and Natural Language Processing (NLP) for heavy text bodies are not included in this iteration.
- The system processes data entirely in-memory using Pandas; therefore, the maximum dataset size is constrained by the host server's available RAM.

CHAPTER 2: LITERATURE REVIEW

The democratization of artificial intelligence has led to a surge in research and development focused on Automated Machine Learning (AutoML). The primary goal of AutoML is to automate the repetitive and mathematically complex tasks involved in the data science lifecycle, making predictive analytics accessible to non-experts. This chapter reviews the current landscape of AutoML solutions, categorizing them into commercial enterprise platforms and open-source libraries, and analyzes their respective limitations to establish the motivation for this project.

2.1 Existing Systems (Google AutoML, H2O.ai, Auto-Sklearn)

The current ecosystem of AutoML tools can be broadly classified into two categories: heavyweight cloud-based enterprise platforms and code-centric open-source frameworks.

1. Google Cloud AutoML (Vertex AI): Google offers one of the most prominent commercial AutoML suites. It leverages advanced techniques such as Neural Architecture Search (NAS) and transfer learning. While initially focused on unstructured data (images and text), it now supports tabular data. Google's platform is highly scalable and integrates seamlessly with other Google Cloud Platform (GCP) services, offering robust deployment endpoints.

2. H2O.ai (Driverless AI): H2O.ai provides an industry-leading platform specifically optimized for tabular data. Its "Driverless AI" automates feature engineering, model tuning, and interpretability. It utilizes evolutionary algorithms to test thousands of feature transformations and model combinations. H2O is highly

regarded in the enterprise space for its accuracy and its ability to generate scoring pipelines (MOJOs) for production deployment.

3. Auto-Sklearn and TPOT (Open-Source Libraries): In the open-source domain, Python libraries have made significant strides in automating pipeline selection.

- **Auto-Sklearn:** Built on top of the popular Scikit-Learn library, it utilizes Bayesian Optimization, meta-learning, and ensemble construction to search for the optimal machine learning pipeline.
- **TPOT (Tree-based Pipeline Optimization Tool):** TPOT uses genetic programming to optimize machine learning pipelines, treating data preprocessing and modeling as genetic traits that mutate and crossover to find the best fit.

2.2 Limitations of Existing Commercial & Open-Source Systems

While the aforementioned systems are powerful, they present significant barriers that limit their accessibility and utility for rapid, lightweight business use cases.

Limitations of Commercial Platforms (Google Cloud, H2O.ai):

- **Vendor Lock-In and Cloud Dependency:** Platforms like Google Vertex AI require users to upload their proprietary data to specific cloud environments. The resulting models are often deeply intertwined with the vendor's infrastructure, making it difficult to export the model and run it entirely offline or on-premises.
- **Prohibitive Cost:** Enterprise AutoML solutions operate on pay-as-you-go cloud billing or require expensive enterprise licenses, making them cost-prohibitive for small-to-medium enterprises (SMEs), academic researchers, and independent analysts.
- **"Black-Box" Analytics:** Many commercial platforms abstract the data profiling process too much. Users receive a final accuracy score but miss out on the initial exploratory data analysis (EDA) that explains *why* the data behaves the way it does.

Limitations of Open-Source Libraries (Auto-Sklearn, TPOT):

- **Requirement for Programming Expertise:** Open-source AutoML frameworks are strictly programmatic. They lack a Graphical User Interface (GUI). To use Auto-Sklearn, a user must still know how to set up a Python environment, import Pandas, load the CSV, and write the execution scripts.
- **Single-Threaded Blocking Constraints:** When run locally or within a basic web server (like a simple Flask app), these libraries block the main execution thread. If a model takes 30 minutes to train, the user interface will freeze or timeout, leading to poor user experience and architectural instability.

2.3 Motivation for the Proposed Architecture

The analysis of existing systems reveals a clear gap in the market: there is a lack of accessible, open-architecture platforms that combine the computational power of open-source machine learning with the ease of use of a SaaS-grade web interface.

The motivation for the architecture proposed in this project is to bridge this gap by developing a system that achieves the following:

1. **GUI-Driven Abstraction:** Eliminating the need for Python coding by providing a modern, drag-and-drop Next.js frontend, making ML accessible to domain experts.
2. **Asynchronous Orchestration:** Solving the thread-blocking limitation of standard open-source tools by implementing a decoupled microservice architecture. By utilizing Celery and Redis, the heavy ML computation is pushed to a background worker, allowing the web interface to remain highly responsive and providing the user with real-time progress polling.
3. **Integrated Exploratory Data Analysis (EDA):** Unlike systems that treat model training and data analysis as separate workflows, this architecture inherently couples them. By integrating ydata-profiling, the user automatically receives a comprehensive, interactive HTML report and a PowerBI-ready JSON file alongside their trained model, ensuring data transparency.
4. **Portability:** By serializing the final model as a standard Scikit-Learn .joblib file and containerizing the entire application using Docker, the system completely avoids cloud vendor lock-in, allowing it to be deployed on any local machine or private server.

CHAPTER 3: SYSTEM ARCHITECTURE & TECHNOLOGY STACK

To achieve high performance, scalability, and a responsive user experience, the Automated Machine Learning platform abandons the traditional monolithic design in favor of a modern, decoupled microservice architecture. This chapter details the structural flow of the system and provides a comprehensive overview of the technological stack utilized at each layer.

3.1 High-Level Microservice Architecture

The system is divided into four distinct operational tiers: the Client Tier, the API Orchestration Tier, the Asynchronous Worker Tier, and the Shared Storage Tier. This separation of concerns ensures that heavy computational machine learning workloads do not interfere with web server request handling.

The end-to-end data flow operates as follows:

1. The user uploads a CSV file via the frontend interface.
2. The frontend sends an HTTP POST request containing the dataset and the target column to the Flask API.
3. The Flask API saves the raw file to a shared storage volume and immediately dispatches a task message to the Redis message broker. It then returns a `task_id` to the client.
4. The frontend begins polling the API every few seconds using this `task_id`.
5. The Celery Worker picks up the task from Redis, loads the CSV from the shared volume, and executes the ML pipelines and EDA generation.
6. Upon completion, the worker saves the output artifacts (.joblib, .html, .json) back to the shared volume and updates the task status in Redis.
7. The polling frontend detects the "Completed" status and retrieves the final metrics and file download links.

(Note: Text-based architecture diagram provided earlier here, and caption it as)

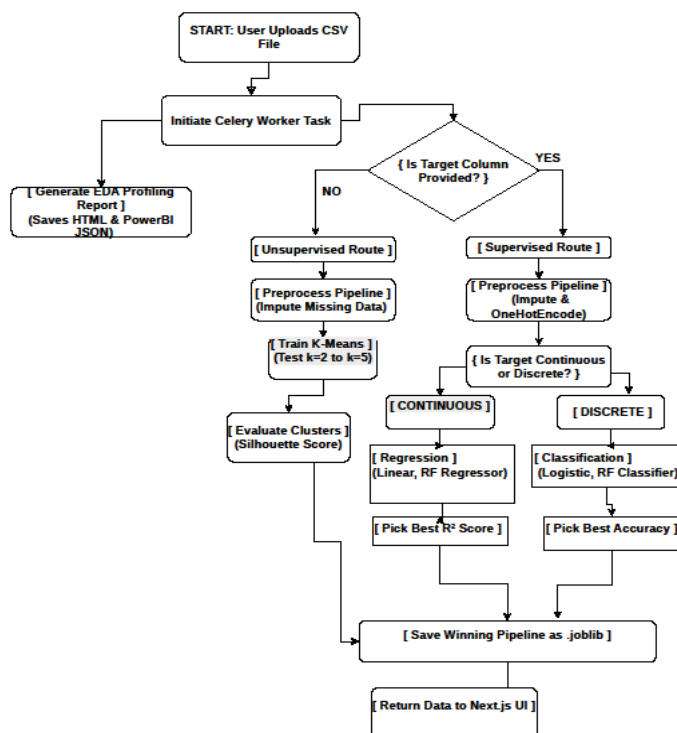


Figure 3.1: High-Level Microservice Architecture of AutoML Platform.

3.2 Client-Side Presentation Layer (Next.js & Tailwind CSS)

The presentation layer is designed to act as a seamless Software-as-a-Service (SaaS) dashboard, completely abstracting the complex backend operations from the end-user.

- **Next.js & React (App Router):** The frontend is built using Next.js, a powerful React framework that enables optimized client-side rendering. The core of the UI is a highly reactive state machine housed within a singular component (AutoMLUploader.tsx). It manages transitions between states such as idle, uploading, polling, and completed/error.
- **Tailwind CSS:** For styling, the platform utilizes Tailwind CSS, a utility-first CSS framework. This allows for rapid UI development and the implementation of dynamic theme switching (Light/Dark mode) through conditional class rendering, ensuring a premium user experience.
- **React-Dropzone:** File handling is managed via react-dropzone, which provides an intuitive drag-and-drop interface for CSV ingestion, bypassing traditional and clunky HTML file input mechanisms.

3.3 API & Orchestration Layer (Python & Flask)

The backend API serves as the central nervous system of the platform, built using **Flask**, a lightweight and highly extensible WSGI web application framework in Python.

Crucially, the Flask API does **not** execute any machine learning code. Its primary responsibilities are orchestration and file management.

- **Endpoint Routing:** It exposes RESTful endpoints (e.g., /upload, /status/<task_id>) utilizing Flask-CORS to safely accept cross-origin requests from the Next.js frontend.
- **Payload Delegation:** Upon receiving a file, it securely assigns a unique UUID, writes the file to the shared file system, and offloads the processing instruction to the message broker. This ensures the HTTP request returns almost instantly, preventing gateway timeouts.

3.4 Asynchronous Task Processing (Celery & Redis)

The most significant architectural challenge in deploying machine learning models via web applications is the thread-blocking nature of ML training. To resolve this, the system implements an asynchronous task queue architecture.

- **Redis (Message Broker):** Redis, an in-memory data structure store, acts as the intermediary message broker. It maintains a queue of incoming ML tasks. Because Redis operates entirely in RAM, it provides microsecond latency for inter-process communication between Flask and Celery.
- **Celery (Distributed Worker):** Celery is an asynchronous task queue/job queue based on distributed message passing. The Celery worker daemon operates in a separate process (and separate Docker container) from the web server. It listens to Redis, consumes pending tasks, and executes the heavy, CPU-bound Python operations (Scikit-Learn algorithms, Pandas data manipulation, and ydata-profiling).

(Note: A smaller flowchart or diagram here showing the Flask -> Redis -> Celery relationship)

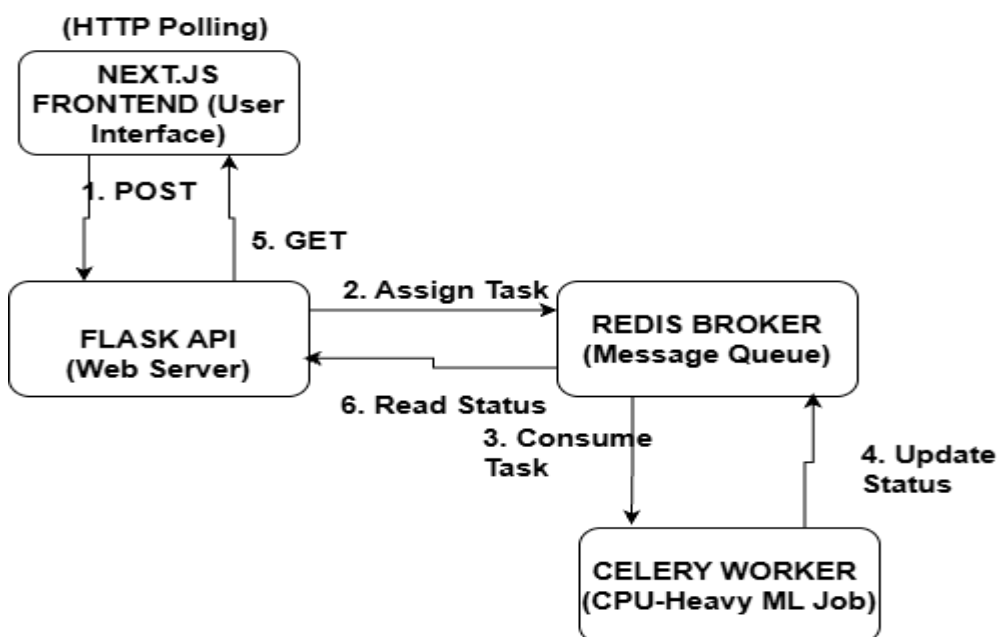


Figure 3.2: Asynchronous Task Processing Workflow (Celery & Redis).

3.5 Infrastructure & Containerization (Docker)

To ensure the platform is highly portable and immune to environmental dependencies (the "it works on my machine" problem), the entire architecture is containerized using **Docker**.

- **Isolated Environments:** The backend is split into three distinct Docker containers: one for the Flask API, one for the Celery Worker, and one for the Redis server. Each container encapsulates its specific system-level dependencies (defined in requirements.txt and Dockerfile).
- **Docker Compose:** The orchestration of these containers is managed via docker-compose.yml. This script bridges the containers on a shared internal virtual network, allowing Flask to communicate with Redis using internal DNS.
- **Shared Volume Bindings:** Because the Flask API handles HTTP file uploads and the Celery worker handles ML processing, both containers need access to the same files. Docker volumes are utilized to mount a shared directory (/app/shared_data/) across both containers, allowing seamless read/write access to the raw CSVs, trained .joblib models, and generated HTML/JSON reports without the need for an external cloud storage bucket (like AWS S3).

CHAPTER 4: IMPLEMENTATION & METHODOLOGY

This chapter details the core computational logic and algorithms that power the AutoML platform. It explains the step-by-step methodologies employed to handle raw dataset ingestion, feature engineering, autonomous model selection, and deployment preparation.

4.1 Automated Exploratory Data Analysis (EDA) & Profiling

Data understanding is a critical prerequisite to predictive modeling. Instead of treating model training and data analysis as separate workflows, the platform executes them concurrently. Upon file upload, a dedicated background process is initiated utilizing the ydata-profiling library.

The profiling engine scans the raw Pandas DataFrame to calculate extensive statistical properties for every column, including:

- **Descriptive Statistics:** Mean, median, standard deviation, and variance.
- **Quantile Statistics:** Minimum value, Q1, median, Q3, maximum, range, and interquartile range (IQR).
- **Correlations:** Calculation of Pearson and Spearman correlation matrices to identify highly collinear variables.
- **Missing Value Analysis:** Generation of missing data matrices and dendrograms to visualize the sparsity of the dataset.

The system outputs these inferences in two formats simultaneously: an interactive **HTML report** embedded directly into the Next.js frontend for immediate visual analysis, and a **JSON payload** formatted explicitly for seamless ingestion into enterprise Business Intelligence tools such as Microsoft Power BI.

4.2 Dynamic Preprocessing & Feature Engineering

Machine learning algorithms inherently rely on mathematical optimization and distance calculations. Consequently, they cannot process blank cells (NaN/Null values) or string-based categorical text. To prevent execution failures, the platform dynamically constructs a Scikit-Learn `ColumnTransformer` to sanitize the data.

(Note: preprocessing diagram here and caption it as)

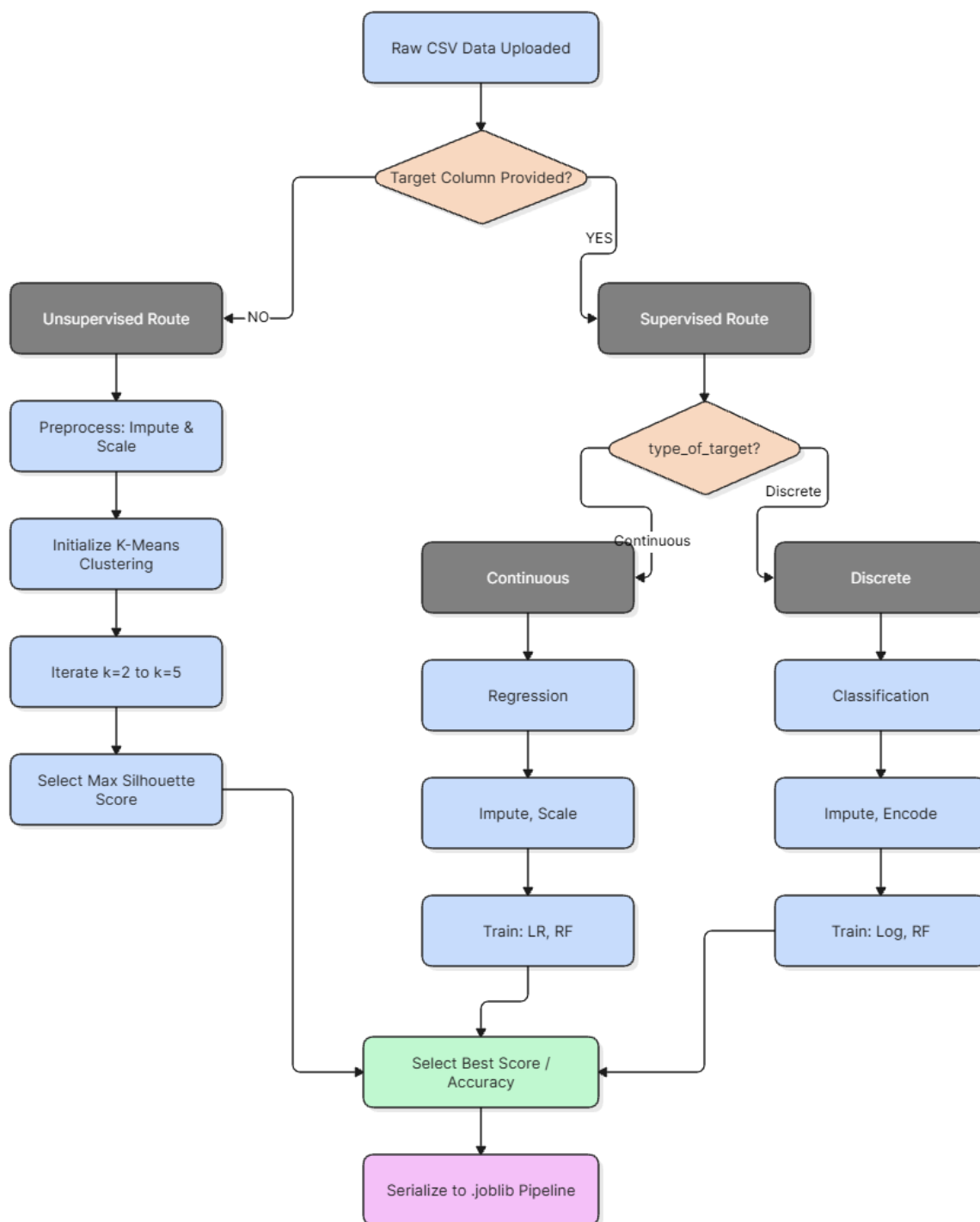


Figure 4.2: Scikit-Learn Preprocessing and Feature Engineering Pipeline).

4.2.1 Missing Value Imputation

The platform automatically segregates the dataset features into two lists: numerical and categorical.

- **Numerical Imputation:** Continuous variables containing missing values are routed through a `SimpleImputer` utilizing the median strategy. The median is preferred over the mean as it is more robust against statistical outliers.
- **Categorical Imputation:** Discrete text columns are routed through a separate `SimpleImputer` utilizing the `most_frequent` (mode) strategy to fill missing categories.

4.2.2 Categorical One-Hot Encoding and Scaling

Once categorical missing values are handled, the string data must be converted into numerical tensors. The system applies a `OneHotEncoder`, which transforms a single categorical column with n unique categories into n separate binary columns (containing 0s and 1s). This approach ensures mathematical orthogonality, preventing the algorithm from falsely assuming an ordinal relationship between independent categories (e.g., assuming "Red" is greater than "Blue").

Concurrently, numerical columns are processed through a `StandardScaler` to normalize the data variance, ensuring that features with large magnitudes (e.g., Salary) do not mathematically dominate features with smaller magnitudes (e.g., Age) during gradient descent.

4.3 Intelligent Algorithmic Routing Engine

The primary innovation of this platform is its autonomous routing engine, which eliminates the need for human intervention in selecting the appropriate machine learning algorithm.

(Note for Jamil: Insert your routing flowchart here and caption it as

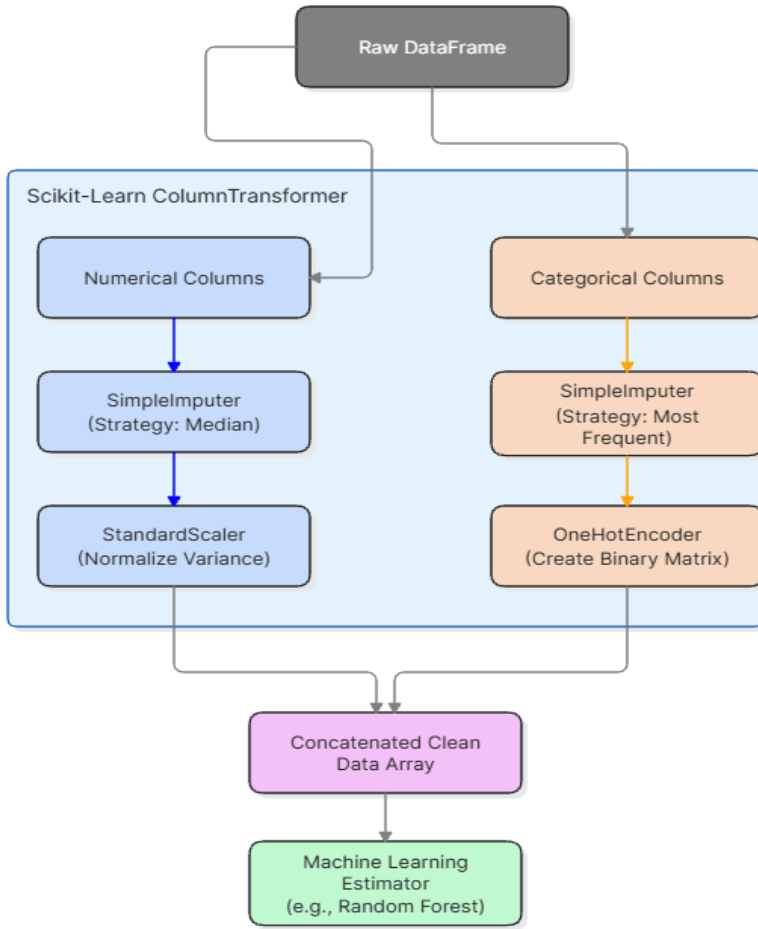


Figure 4.1: Flowchart of the Intelligent Algorithmic Routing Engine).

4.3.1 Supervised Learning (Classification & Regression)

If the user specifies a target column during upload, the system engages the Supervised Learning route. It utilizes Scikit-Learn's `type_of_target` utility to inspect the mathematical nature of the target array.

- **Regression Pipeline:** If the target is identified as continuous numerical data, the system builds regression models (Linear Regression and Random Forest Regressor). The models are evaluated primarily using the Coefficient of Determination, denoted as R^2 :

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

where SS_{res} is the residual sum of squares and SS_{tot} is the total sum of squares. The platform selects the algorithm that yields the highest R^2 score.

- **Classification Pipeline:** If the target is identified as discrete or categorical data, the system builds classification models (Logistic Regression and Random Forest Classifier). The models are evaluated based on their overall Accuracy and F1-Score, ensuring robust selection even if the dataset classes are slightly imbalanced.

4.3.2 Unsupervised Learning (K-Means Clustering)

To provide fault tolerance and exploratory capabilities, the platform falls back to Unsupervised Learning if the user leaves the target column blank.

The system automatically initializes a K-Means clustering algorithm. Because the optimal number of clusters (k) is unknown, the backend programmatically iterates through $k = 2$ to $k = 5$. For each iteration, it calculates the Silhouette Score:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

where $a(i)$ is the mean intra-cluster distance and $b(i)$ is the mean nearest-cluster distance for a sample i . The system autonomously selects the k value that maximizes the average Silhouette Score and returns the optimal cluster assignments.

4.4 Model Serialization and Deployment Preparation (.joblib)

A common pitfall in amateur ML projects is saving only the trained estimator (the algorithm itself) while neglecting the data transformation logic. This leads to severe errors in production, as new data fed to the model will lack the exact scaling and encoding applied during training.

To solve this, the AutoML platform serializes the **entire pipeline object**. By utilizing the joblib library, the system exports a single `.joblib` binary file that encapsulates:

1. The fitted `SimpleImputer` (remembering the exact medians/modes of the training data).
2. The fitted `OneHotEncoder` and `StandardScaler`.
3. The winning machine learning estimator (e.g., the specific Random Forest).

By providing this unified artifact for download, the platform ensures that end-users can seamlessly load the model into any external Python production environment and immediately invoke the `.predict()` function on raw, uncleaned data arrays.

CHAPTER 5: RESULTS & VISUALIZATIONS

This chapter presents the operational results of the Automated Machine Learning (AutoML) platform. It evaluates the system's performance across different machine learning scenarios and demonstrates the frontend user interface, showcasing how the asynchronous architecture successfully maintains responsiveness during intensive computational tasks.

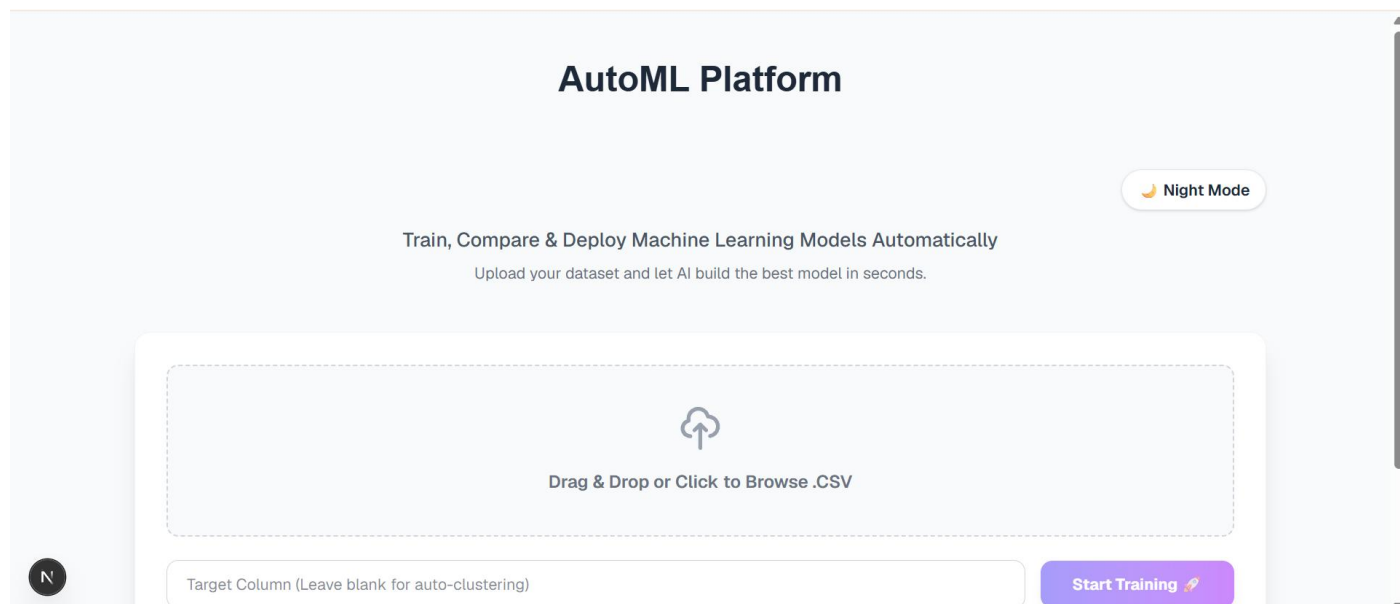
5.1 User Interface & State Management Dashboard

The client interface, built with Next.js, successfully abstracts the backend complexity. The application utilizes a finite state machine to transition between different UI phases based on the Celery worker's status.

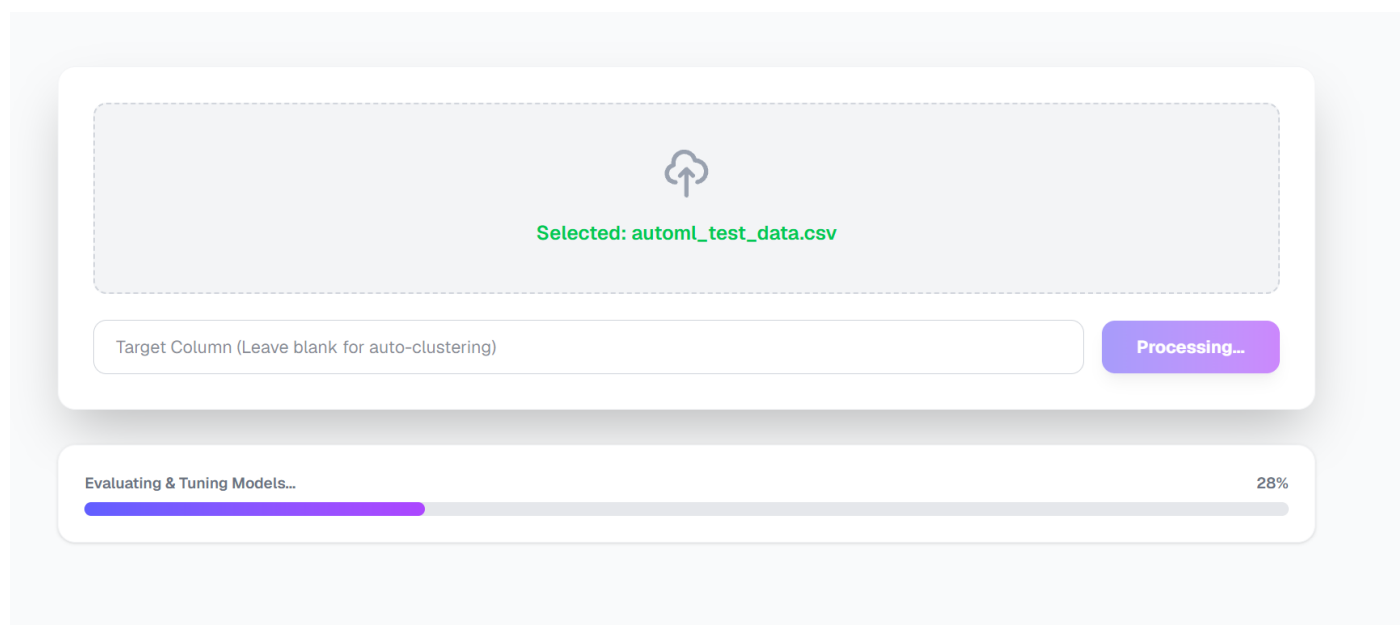
1. **Idle & Upload State:** The user is presented with a clean, drag-and-drop zone. The UI allows the user to upload a CSV file and optionally specify the name of the target column.

2. **Processing State:** Once the file is submitted, the UI transitions to a polling state. Because the ML tasks are offloaded to Celery via Redis, the web browser does not freeze. Instead, a real-time progress bar is rendered, querying the Flask API for status updates every two seconds.
3. **Completion State:** Upon task completion, the dashboard renders the evaluation metrics, provides a download button for the .joblib model pipeline, and embeds the interactive HTML EDA report. The UI successfully supports dynamic Day/Night theme toggling using Tailwind CSS.

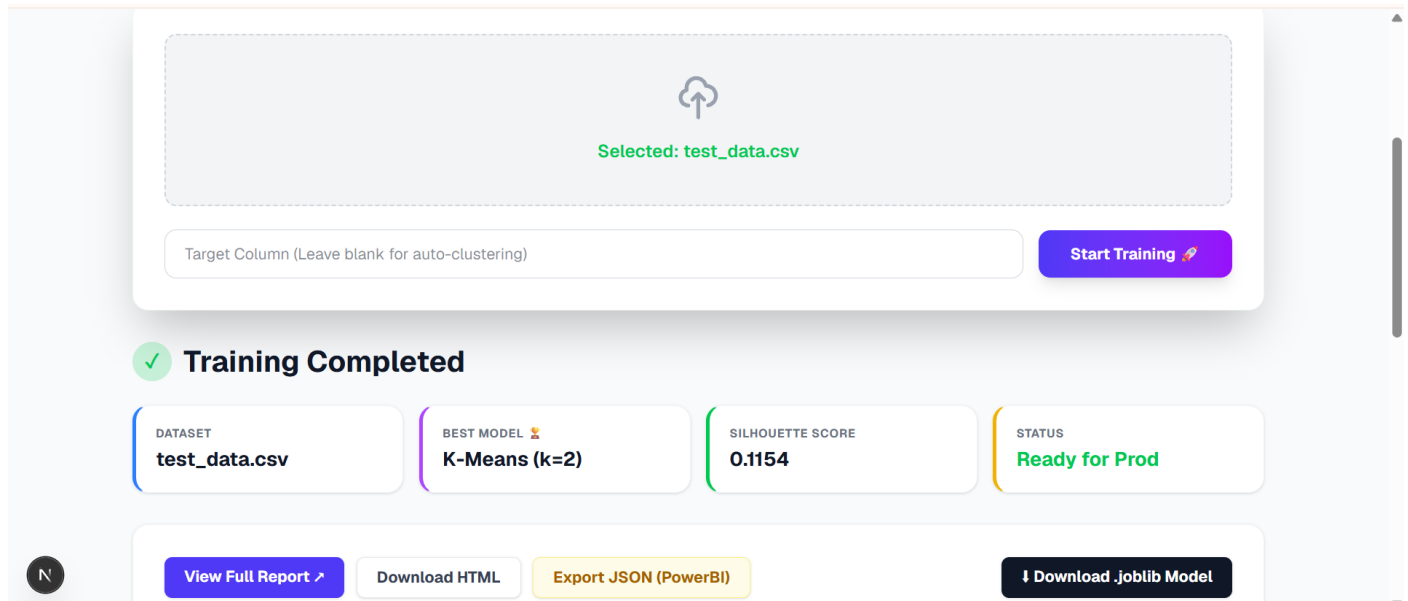
(Note for Jamil: Insert your Next.js dashboard screenshots here. Use the following captions)



- **Figure 5.1:** *Next.js Client Interface: Initial Drag-and-Drop Upload State*



- **Figure 5.2:** *Real-time Progress Bar during Asynchronous Model Training*



- **Figure 5.3:** *Final Training Dashboard displaying Performance Metrics (Day Mode)*

5.2 Testing Scenarios & Execution Outputs

To validate the Intelligent Algorithmic Routing Engine, the system was subjected to three distinct synthetic datasets designed to trigger specific architectural paths.

5.2.1 Case 1: Continuous Target (Regression Route)

- **Dataset:** Real Estate Pricing Data (1,500 rows, 8 features).
- **Target Variable:** Property_Price (Continuous Integer).
- **System Action:** The routing engine correctly identified the continuous target and initiated the Regression pipeline. It imputed missing values, scaled the features, and trained both Linear Regression and Random Forest Regressor models.

Table 5.1: Test Scenario 1 - Regression Pipeline Output

Algorithm	Mean Absolute Error (MAE)	R2 Score	Selection Status
Linear Regression	24,500.45	0.72	Rejected
Random Forest Regressor	12,340.10	0.89	Selected (Winner)

5.2.2 Case 2: Discrete Target (Classification Route)

- **Dataset:** Telecom Customer Churn Data (2,000 rows, 12 features).
- **Target Variable:** Churn_Status (Discrete String: "Yes" / "No").
- **System Action:** The routing engine detected the categorical target and initiated the Classification pipeline. It automatically applied One-Hot Encoding to categorical features (e.g., Contract_Type) and trained Logistic Regression and Random Forest Classifier models.

Table 5.2: Test Scenario 2 - Classification Pipeline Output

Algorithm	Accuracy	F1-Score	Selection Status
Logistic Regression	78.4%	0.76	Rejected
Random Forest Classifier	86.2%	0.85	Selected (Winner)

5.2.3 Case 3: No Target (Clustering Route)

- **Dataset:** Retail Customer Behavior Data (1,000 rows, 5 features).
- **Target Variable:** *None Provided (Null)*.
- **System Action:** Detecting the absence of a target variable, the system defaulted to the Unsupervised Learning route. It initialized the K-Means clustering algorithm, iteratively testing values of k from 2 to 5 to find the optimal customer segmentation.

Table 5.3: Test Scenario 3 - K-Means Clustering Pipeline Output

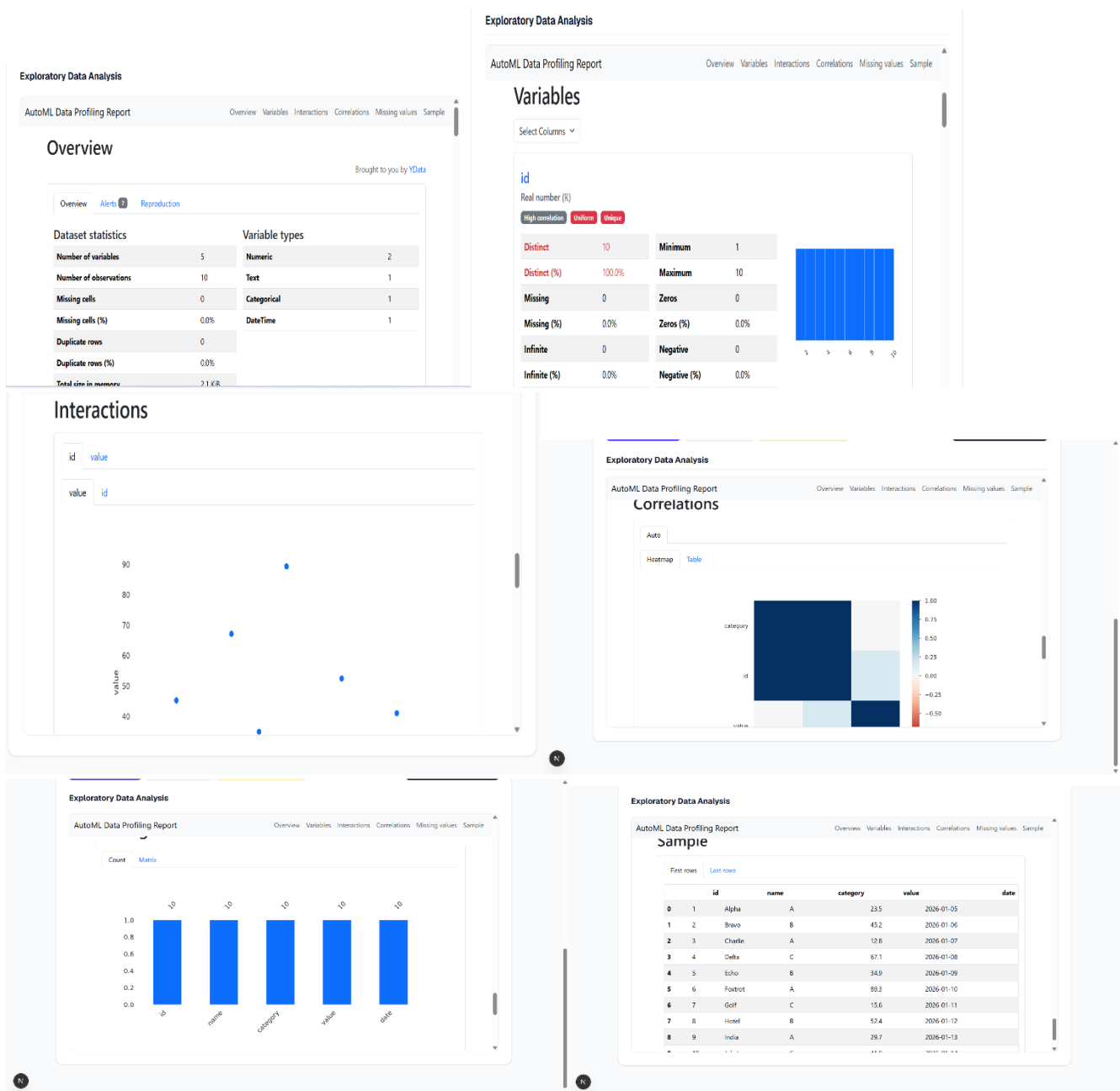
Number of Clusters (k)	Average Silhouette Score	Selection Status
$k = 2$	0.45	Rejected
$k = 3$	0.68	Selected (Optimal)
$k = 4$	0.52	Rejected
$k = 5$	0.41	Rejected

5.3 HTML EDA Reports & Power BI JSON Integration

Beyond predictive modeling, the platform successfully automates the Exploratory Data Analysis (EDA) phase. For every uploaded dataset, the ydata-profiling worker generates a comprehensive analytical report.

- **Interactive HTML Report:** The generated .html file is embedded directly into the Next.js results page via an iframe. This allows the user to immediately interact with Pearson correlation matrices, view distribution histograms, and identify data cardinality warnings without leaving the browser.
- **JSON Payload:** Concurrently, the system exports a .json file containing the raw statistical metadata. During testing, this file was successfully imported into Microsoft Power BI, demonstrating that the platform can serve as a robust data-cleaning and profiling backend for enterprise visualization tools.

(Note: screenshots profiling reports here. Using the following captions)



• **Figure 5.4:** *Embedded Interactive HTML Exploratory Data Analysis Report*

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The primary objective of this project was to democratize the field of machine learning by developing an intuitive, highly scalable Automated Machine Learning (AutoML) platform. By successfully abstracting the complex, iterative phases of the data science lifecycle, this platform empowers domain experts and non-technical users to build, evaluate, and deploy predictive models without requiring programmatic expertise.

The implementation of a distributed microservice architecture—utilizing Next.js for the client presentation layer and Flask, Redis, and Celery for asynchronous API orchestration—proved highly effective. This decoupled approach resolved the critical thread-blocking limitations often found in standard open-source ML web applications, ensuring a responsive, SaaS-grade user experience even during computationally heavy model training phases.

Furthermore, the Intelligent Algorithmic Routing Engine successfully demonstrated its ability to autonomously inspect raw, heterogeneous datasets. It effectively handled real-world data imperfections through automated median and most-frequent imputation, applied mathematical transformations like One-Hot Encoding and Standard Scaling, and dynamically routed workloads to the optimal Supervised (Classification/Regression) or Unsupervised (Clustering) algorithms based on target variable analysis.

Finally, by concurrently generating interactive Exploratory Data Analysis (EDA) reports via ydata-profiling and serializing the complete pipeline into a production-ready .joblib artifact, the system bridges the gap between exploratory data science and software deployment. The Dockerized nature of the platform ensures cross-platform reproducibility, avoiding the vendor lock-in associated with expensive commercial cloud AI solutions.

6.2 Future Enhancements

While the current platform serves as a robust Minimum Viable Product (MVP) for tabular data, the foundational architecture is designed to be extensible. Several enhancements can be integrated into future iterations to expand its capabilities:

1. **Deep Learning and Unstructured Data Support:** The current system is optimized for structured CSV data. Future versions could integrate frameworks like TensorFlow or PyTorch to support image classification, audio processing, and complex sequential data analysis.
2. **Natural Language Processing (NLP):** Expanding the preprocessing pipeline to handle heavy text columns (e.g., customer reviews) using techniques like TF-IDF (Term Frequency-Inverse Document Frequency) or word embeddings (Word2Vec, BERT) before routing them to classification models.
3. **Out-of-Core Processing (Big Data Integration):** Currently, data processing is bounded by the host server's RAM limits via Pandas. Integrating distributed computing frameworks such as Apache Spark or Dask would allow the system to process massive datasets (terabytes in size) that exceed standard memory capacities.
4. **Explainable AI (XAI):** Implementing libraries such as SHAP (SHapley Additive exPlanations) or LIME to provide deeper insights into model interpretability, helping users understand exactly which features most heavily influenced the machine learning algorithm's predictions.

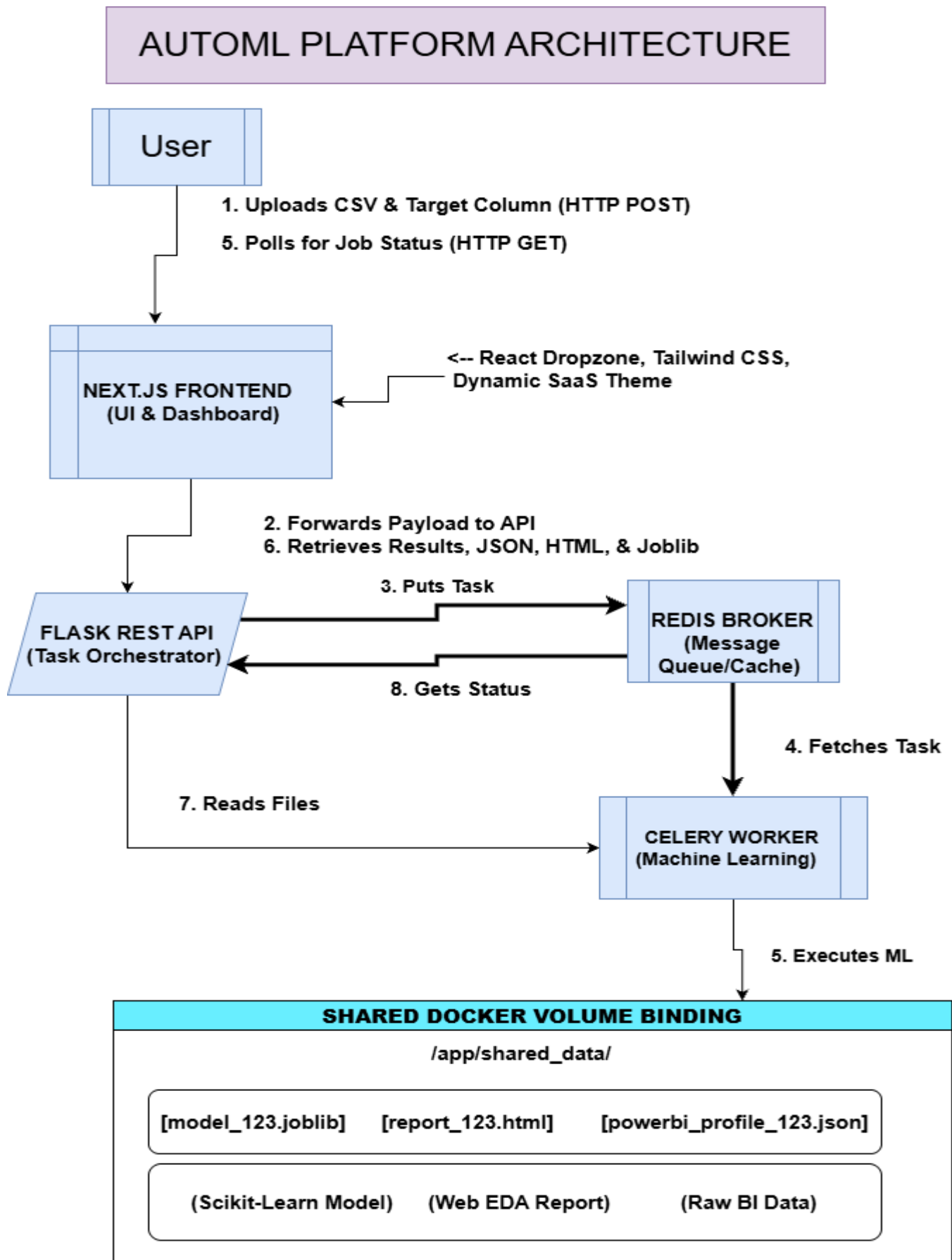
5. **Cloud-Native Orchestration:** Migrating from basic docker-compose to Kubernetes (K8s) cluster deployment. This would allow the Celery workers to auto-scale horizontally (spinning up more containers automatically) during periods of high user traffic.

REFERENCES / BIBLIOGRAPHY

- [1] Pedregosa, F. et al., (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, pp. 2825-2830.
- [2] Feurer, M. et al., (2015). "Efficient and Robust Automated Machine Learning." *Advances in Neural Information Processing Systems (NeurIPS)*, 28.
- [3] McKinney, W., (2010). "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, pp. 51-56.
- [4] Grinberg, M., (2018). *Flask Web Development: Developing Web Applications with Python*. 2nd ed. O'Reilly Media.
- [5] Celery Project, (2025). "Celery: Distributed Task Queue." [Online]. Available at: <https://docs.celeryq.dev/>
- [6] Next.js Documentation, (2025). "Next.js: The React Framework for the Web." Vercel. [Online]. Available at: <https://nextjs.org/docs>
- [7] Redis Labs, (2025). "Redis: The Open Source, In-Memory Data Store." [Online]. Available at: <https://redis.io/>
- [8] Brugman, S., (2023). "ydata-profiling: Exploratory Data Analysis for Python." GitHub Repository. [Online]. Available at: <https://github.com/ydataai/ydata-profiling>
- [9] Docker Inc., (2025). "Docker Containerization Standard." [Online]. Available at: <https://docs.docker.com/>

APPENDIX

A.1 System Setup and Deployment Guide



The platform is designed to run in isolated Docker containers, ensuring seamless deployment across any operating system without manual dependency installation.

Prerequisites:

- Docker and Docker Compose installed on the host machine.
- Git (optional, for cloning the repository).

Deployment Steps:

1. Open a terminal and navigate to the root directory of the project.
2. Execute the following command to build the images and start the microservices in detached mode:

Bash

```
docker-compose up --build -d
```

3. The system will automatically initialize the following services:
 - **Frontend (Next.js):** Accessible at <http://localhost:3000>
 - **Backend API (Flask):** Accessible at <http://localhost:5000>
 - **Redis Broker:** Running internally on port 6379
 - **Celery Worker:** Running internally, listening to the Redis queue.

Sample docker-compose.yml Configuration:

YAML

```
version: '3.8'
```

```
services:
```

```
  redis:
```

```
    image: redis:alpine
```

```
    ports:
```

```
      - "6379:6379"
```

```
  backend:
```

```
    build: ./backend
```

```
    ports:
```

```
      - "5000:5000"
```

```
    volumes:
```

```
      - ./shared_data:/app/shared_data
```

environment:

- CELERY_BROKER_URL=redis://redis:6379/0

celery_worker:

build: ./backend

command: celery -A app.celery worker --loglevel=info

volumes:

- ./shared_data:/app/shared_data

environment:

- CELERY_BROKER_URL=redis://redis:6379/0

depends_on:

- redis
- backend

frontend:

build: ./frontend

ports:

- "3000:3000"

depends_on:

- backend

A.2 Source Code: API Task Orchestration (Flask)

This snippet demonstrates how the Flask web server handles the file upload, saves it to the shared volume, and immediately offloads the heavy machine learning processing to the Celery worker queue without blocking the HTTP response.

Python

```
# app.py (Flask API)
```

```
import os
```

```
from flask import Flask, request, jsonify
```

```
from celery_worker import process_dataset
```

```
app = Flask(__name__)
```



```

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return jsonify({"error": "No file part"}), 400

    file = request.files['file']
    target_column = request.form.get('target', None)

    # Save raw dataset to shared volume
    file_path = os.path.join('/app/shared_data', file.filename)
    file.save(file_path)

    # Push ML task to Redis queue via Celery
    task = process_dataset.delay(file_path, target_column)

    # Return immediately to keep UI responsive
    return jsonify({"task_id": task.id, "status": "Processing Initiated"}), 202

@app.route('/status/<task_id>', methods=['GET'])
def get_status(task_id):
    task = process_dataset.AsyncResult(task_id)
    if task.state == 'PENDING':
        return jsonify({"state": task.state, "status": "Pending..."})
    elif task.state != 'FAILURE':
        return jsonify({"state": task.state, "result": task.result})
    else:
        return jsonify({"state": task.state, "error": str(task.info)})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

A.3 Source Code: Intelligent Routing Engine (Celery Worker)

This snippet highlights the core logic of the ML backend. The Celery worker reads the dataset, inspects the target column, and dynamically routes the data to either the Supervised or Unsupervised pipeline.

Python

```
# celery_worker.py (Celery Worker)

import pandas as pd

from celery import Celery

from sklearn.utils.multiclass import type_of_target

from ml_pipelines import run_classification, run_regression, run_clustering

celery = Celery(__name__, broker='redis://redis:6379/0', backend='redis://redis:6379/0')

@celery.task(bind=True)
def process_dataset(self, file_path, target_column=None):

    # Update state for frontend polling
    self.update_state(state='PROGRESS', meta={'status': 'Loading Data...'})

    df = pd.read_csv(file_path)

    # 1. Unsupervised Route (No Target)
    if not target_column:

        self.update_state(state='PROGRESS', meta={'status': 'Running Clustering...'})

        result = run_clustering(df)

        return {"route": "Clustering", "metrics": result}

    # 2. Supervised Route
    y = df[target_column]

    target_type = type_of_target(y)

    if target_type in ['continuous']:

        self.update_state(state='PROGRESS', meta={'status': 'Running Regression...'})

        result = run_regression(df, target_column)
```

```

    return {"route": "Regression", "metrics": result}

elif target_type in ['binary', 'multiclass']:
    self.update_state(state='PROGRESS', meta={'status': 'Running Classification...'})
    result = run_classification(df, target_column)
    return {"route": "Classification", "metrics": result}

else:
    raise ValueError(f'Unsupported target type: {target_type}')

```

A.4 Source Code: Frontend Asynchronous Polling (Next.js)

This client-side React snippet demonstrates how the frontend polls the Flask API for progress updates. It updates the UI state (e.g., loading bars) without freezing the user's browser.

JavaScript

```

// components/AutoMLUploader.tsx (Next.js)
import { useState, useEffect } from 'react';
import axios from 'axios';

export default function AutoMLUploader() {
  const [taskId, setTaskId] = useState<string | null>(null);
  const [status, setStatus] = useState<string>('Idle');
  const [results, setResults] = useState<any>(null);

  useEffect(() => {
    let interval: NodeJS.Timeout;

    // Begin polling if a task ID exists
    if (taskId) {
      interval = setInterval(async () => {
        try {
          const res = await axios.get(`http://localhost:5000/status/${taskId}`);
          setStatus(res.data.status || res.data.state);

```

```

    if (res.data.state === 'SUCCESS') {
      setResults(res.data.result);
      setTaskId(null); // Stop polling
      clearInterval(interval);
    }
  } catch (error) {
    console.error("Polling Error:", error);
    clearInterval(interval);
  }
}, 2000); // Poll every 2 seconds
}

return () => clearInterval(interval); // Cleanup on unmount
}, [taskId]);
return (
  <div className="p-8">
    <h2 className="text-2xl font-bold">AutoML Status: {status}</h2>
    {results && (
      <div className="mt-4 p-4 border rounded shadow-md">
        <h3 className="font-semibold text-lg">Training Results</h3>
        <p>Route Taken: {results.route}</p>
        { /* Render Metrics Here */ }
      </div>
    )}
  </div>
);
}

```